

CSE 141L Milestone 2

Ryan Goh, A16456998; Ellie Morissette, A15689296; Ryan Vanny, A1672574

Academic Integrity

Your work will not be graded unless the signatures of all members of the group are present beneath the honor code.

To uphold academic integrity, students shall:

- Complete and submit academic work that is their own and that is an honest and fair representation of their knowledge and abilities at the time of submission.
- Know and follow the standards of CSE 141L and UCSD.

Please sign (type) your name(s) below the following statement:

I pledge to be fair to my classmates and instructors by completing all of my academic work with integrity. This means that I will respect the standards set by the instructor and institution, be responsible for the consequences of my choices, honestly represent my knowledge and abilities, and be a community member that others can trust to do the right thing even when no one is watching. I will always put learning before grades, and integrity before performance. I pledge to excel with integrity.

Ryan Goh
Ellie Morissette
Ryan Vanny

0. Team

Ryan Goh, Ryan Vanny, Ellie Morissette

1. Introduction

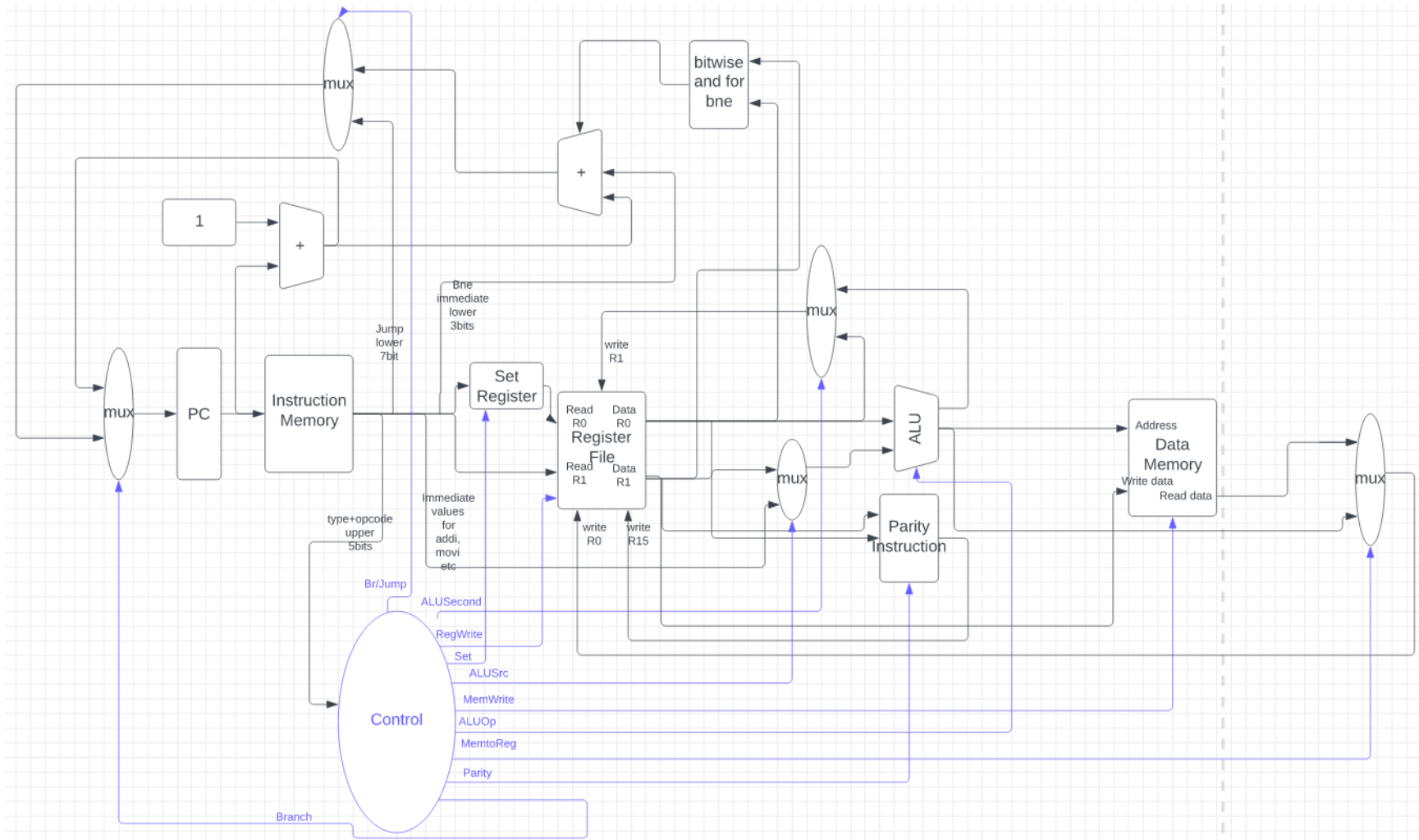
Architecture: AccuRISC

Overall philosophy: Large number of registers relative to instruction length to reduce memory access, assignable accumulator to minimize on bits-per-instruction (only need one argument per instruction)

Specific goals: Efficient use of instruction space (one-argument R-types/I-types, etc)

Classification: Combination of load/store and accumulator philosophies

2. Architectural Overview



3. Machine Specification

Instruction formats

TYPE	FORMAT	CORRESPONDING INSTRUCTIONS
N	1 bit type, 4 bits opcode, rest of instruction ignored	halt, noop
R	1 bit type, 4 bits opcode, 4 bit operand register	add, xor, or, lsr, par, lw, sw, set
I	1 bit type, 4 bits opcode, 4 bit immediate	addi, subi, lsl, movi
J	1 bit type, 1 bits opcode, 7-bit address	j
B	1 bit type, 1 bits opcode, 2 bit operand register, 2 bit operand register, 3 bit branch offset	bne

Operations

NAME	TYPE	BIT BREAKDOWN	EXAMPLE	NOTES
halt = stop execution	N	1 bit type (0), 4 bits opcode (0000), rest of instruction ignored	halt $\Leftrightarrow$ 0_0000_0000 # after halt instruction, execution stops	Any halt instruction halts the machine

noop = noop	N	1 bit type (0), 4 bits opcode (1111), rest of instruction ignored	noop $\Leftrightarrow$ 0_1111_0000 # after noop instruction, do nothing	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed
add = register addition	R	1 bit type (0), 4 bits opcode (0001), 4 bit operand register (XXXX)	# Assume R0 has 0b0000_1111 # Assume R1 has 0b1000_0001 # Assume RA is set to R0 add R1 $\Leftrightarrow$ 0_0001_0001 # after add instruction, R0 now holds 0b1001_0000	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed
xor = register exclusive-or	R	1 bit type (0), 4 bits opcode (0100), 4 bit operand register (XXXX)	# Assume R0 has 0b1111_1111 # Assume R1 has 0b1111_0000 # Assume RA is set to R0 xor R1 $\Leftrightarrow$ 0_0100_0001 # after xor instruction, R0 now holds 0b0000_1111	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed
or = register or	R	1 bit type (0), 4 bits opcode (0101), 4 bit operand register (XXXX)	# Assume R0 has 0b1010_0101 # Assume R1 has 0b1010_1010 # Assume RA is set to R0 or R1 $\Leftrightarrow$ 0_0101_0001	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed

			# after or instruction, R0 now holds 0b1010_1111	
lrsr = logical register-shift right	R	1 bit type (0), 4 bits opcode (0111), 4 bit operand register (XXXX)	# Assume R0 has 0b0111_1111 # Assume R1 has 0b0000_1111 # Assume RA is set to R0 lrsr R1 ⇔ 0_0111_0001 # after lrsr instruction, R0 now holds 0b0011_1111 and R1 now holds 0b1000_0111	Implied first operand and implied destination register are either the most recent register set by set, or R0 if set hasn't been executed This register works by right-shifting RA bit-by-bit into R1 using a LSB carry-out and a MSB carry-in. It being a logical shift, the MSB of RA becomes 0, and the LSB of R1 is lost.
par = parity	R	1 bit type (0), 4 bits opcode (1000), 4 bit operand register (XXXX)	# Assume RA is set to R1 par R0 ⇔ 0_1000_0000 # after par instruction, R15 now holds the parity string (lowest 5 bits P8 P4 P2 P1 P0) of R1 as MSW, R0 as LSW	Implied first operand and implied destination register are either the most recent register set by set, or R0 if set hasn't been executed
lw = load from memory	R	1 bit type (0), 4 bits opcode (1001), 4 bit operand register (XXXX)	# Assume R1 has 0b1111_1111 # Assume RA is set to R0 lw R1 ⇔ 0_1001_0001 # after lw instruction, R0 now holds the value in mem[255]	Implied first operand and implied destination register are either the most recent register set by set, or R0 if set hasn't been executed
sw = store to memory	R	1 bit type (0), 4 bits opcode (1010), 4 bit operand register (XXXX)	# Assume R1 has 0b1111_1111 # Assume RA is set to R0	Implied first operand and implied destination register are either the most recent register set by set,

			<code>sw R1 ⇔ 0_1010_0001</code> # after sw instruction, mem[255] now holds the value in R0	or R0 if <code>set</code> hasn't been executed
addi = immediate addition	I	1 bit type (0), 4 bits opcode (0010), 4 bit immediate (XXXX)	# Assume R0 has 0b0000_1111 # Assume RA is set to R0 <code>addi R1 ⇔ 0_0010_0001</code> # after addi instruction, R0 now holds 0b0001_0000	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed
subi = immediate subtraction	I	1 bit type (0), 4 bits opcode (0011), 4 bit immediate (XXXX)	# Assume R0 has 0b0001_0000 # Assume RA is set to R0 <code>subi R1 ⇔ 0_0011_0001</code> # after subi instruction, R0 now holds 0b0000_1111	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed
lsl = logical shift left	I	1 bit type (0), 4 bits opcode (0110), 4 bit immediate (XXXX)	# Assume R0 has 0b1111_1111 # Assume RA is set to R0 <code>lsl 3 ⇔ 0_0110_0011</code> # after lsl instruction, R0 now holds 0b1111_1000	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed Immediates larger than 0b100 result in the register always being cleared
movi = move immediate to	I	1 bit type (0), 4 bits opcode (1011), 4 bit immediate (XXXX)	# Assume RA is set to R0	Implied first operand and implied destination register are either the

register			<code>movi 7 ⇔ 0_1011_0111</code> # after movi instruction, R0 now holds <code>0b0000_0111</code>	most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed Can only set a register to a value between 0 and 15; to reach the maximum 255 combine <code>movi</code> with <code>lsl</code> and <code>addi</code>
set = set accumulator register	I	1 bit type (0), 4 bits opcode (1100), 4 bit immediate (XXXX)	<code>set R1 ⇔ 0_1100_0001</code> # after set instruction, accumulator (RA) points to R1	Implied first operand and implied destination register are either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed
jump = jump to address	J	1 bit type (1), 1 bit opcode (0), 7 bit immediate (XXXXXXX)	<code>jump 0b1010101 ⇔ 1_0_1010101</code> # after jump instruction, PC = <code>0bXX1010101</code>	
bne = branch if not equal	B	1 bit type (1), 1 bit opcode (1), 4 bit operand register (XXXX), 3 bit immediate (XXX)	# Assume R0 has <code>0b1111_1111</code> # Assume R1 has <code>0b0000_0000</code> # Assume RA is set to R0 <code>bne R1 7 ⇔ 1_1_0001_111</code> # after bne instruction, PC = PC + 7 # Assume R0 has <code>0b1111_1111</code>	Implied first operand is either the most recent register set by <code>set</code> , or R0 if <code>set</code> hasn't been executed

			# Assume R1 has 0b1111_1111 # Assume RA is set to R0 bne R1 7 ⇔ 1_1_0001_111 # RA = R1, no branch occurs	
--	--	--	---	--

Internal Operands

There are 16 registers supported. All of them are general purpose registers, with an option to “set” a register as an implied accumulator register for our operations to store to.

Control Flow (branches)

Branch-if-not-equal is supported, using a 3-bit PC offset (maximum offset of 7). Jump is supported, using a 7-bit absolute address, meaning you can jump anywhere within a range of $PC[-1:8]+[7:0]0$ to $PC[-1:8]+[7:0]1$ (a range of 128 addresses). We do not expect our program counter to exceed this instruction count, but if it does, subsequent jumps isolated from the code they occur in by either bnes or other jumps would be able to work. For bnes of larger than 7, the same technique of subsequent jumps can be employed.

Addressing Modes

Direct, immediate, and indirect addressing modes are supported. Addresses are directly accessed from a register in the case of arithmetic R instructions, and accessed by an immediate for I instructions. We use indirect addressing for sw and lw instructions where an input register contains the desired address in data memory.

Ex:

Set R1

Add R0 #This directly addresses registers R0 and R1 and stores it to R1

Set R1

Addi 3	#This adds immediate value 3 to the contents of R1
Set R0	
Lw R1	#This indirectly addresses the address contained in R1

4. Programmer's Model [Lite]

4.1 How should a programmer think about how your machine operates? Provide a description of the general strategy a programmer should use to write programs with your machine. For example, one could say that the programmer should prioritize loading in the necessary values from memory into as many registers as possible, then perform calculations. Another approach could be loading and writing to memory in between every calculation step. Word limit: 200 words.

A programmer should think of the machine as a load-store-like machine in terms of memory management, but an accumulator in terms of programming style, with the caveat that you have multiple accumulator registers that you can rotate between with a dedicated instruction. Due to the large number of registers (16), a programmer should not need to frequently utilize loads/stores for complex programs.

4.2 Can we copy the instructions/operation from MIPS or ARM ISA? If no, explain why not? How did you overcome this or how do you deal with this in your current design? Word limit: 100 words.

No, we cannot copy the instructions from MIPS or ARM since we are limited to only 9 bits per machine code instruction whereas MIPS and ARM both use 32 bit instructions. We overcame this by keeping the number of instructions used across all three programs to a minimum, and minimizing the number of operands in the instruction by way of the pseudo-accumulator structure. We also have a specific instruction to calculate the parity string to save on program length.

4.3 Will your ALU be used for non-arithmetic instructions (e.g., MIPS or ARM-like memory address pointer calculations, PC relative branch computations, etc.)? If so, how does that complicate your design?

Yes, our ALU will be performing logical shifts. Specifically, our right shift instruction will require two outputs as we want to shift the MSW right by one while also carrying over the LSB of the MSW into the MSB of the LSW. This requires us to have a second output value from the ALU to store both the high and low bytes to separate registers.

5. Individual Component Specification

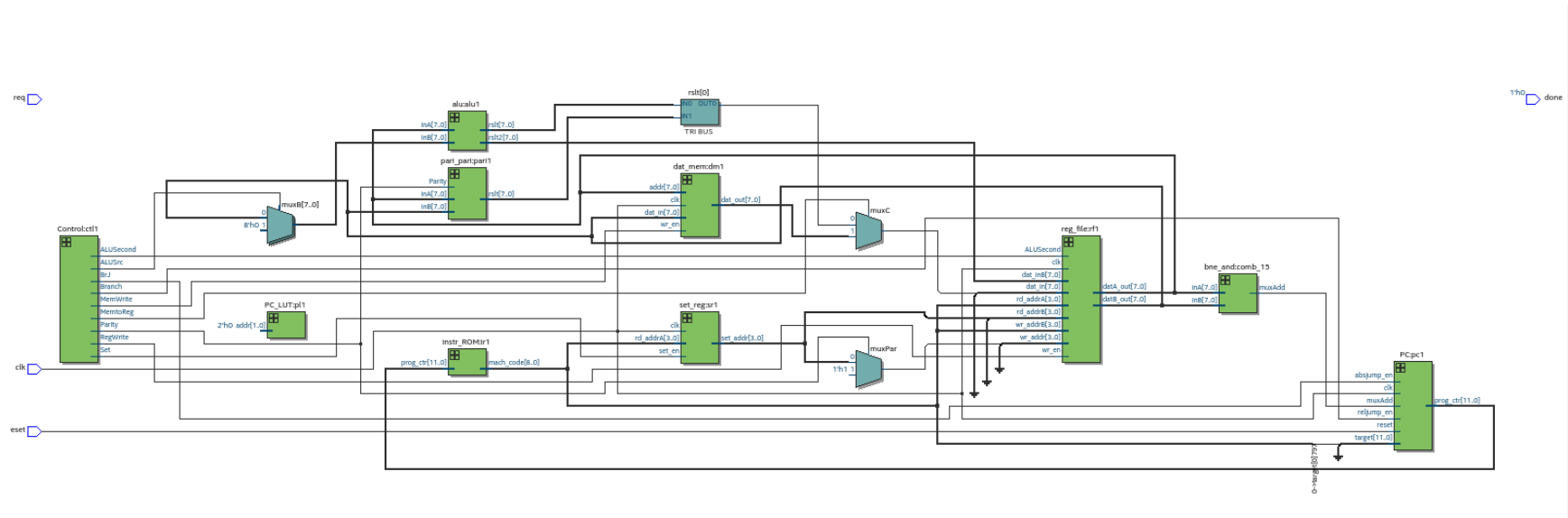
Top Level

Module file name: top_level.sv

Functionality Description

The top level module is the overall circuit for our processor. It provides a high level view of the connections between inputs and outputs of each individual component.

Schematic



Program Counter

Module file name: PC.sv

Module testbench file name: PCtestbench.sv

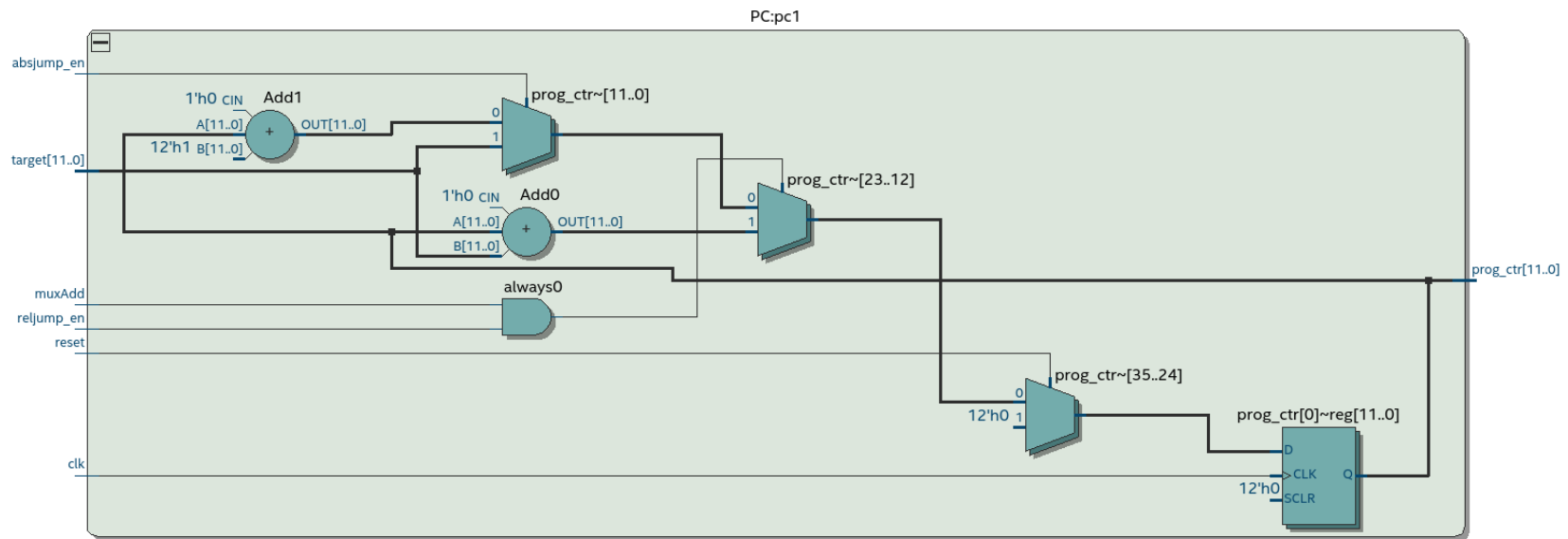
Functionality Description

The PC module handles our standard program counter increment including branch destinations from bne and jump destinations from j. Our bne instruction is a relative jump which adds the target destination to the current PC value, whereas our j instruction is an absolute jump which replaces the PC value. The value of the PC is updated every clock cycle, and receives control logic for resets, branches, and jump flags.

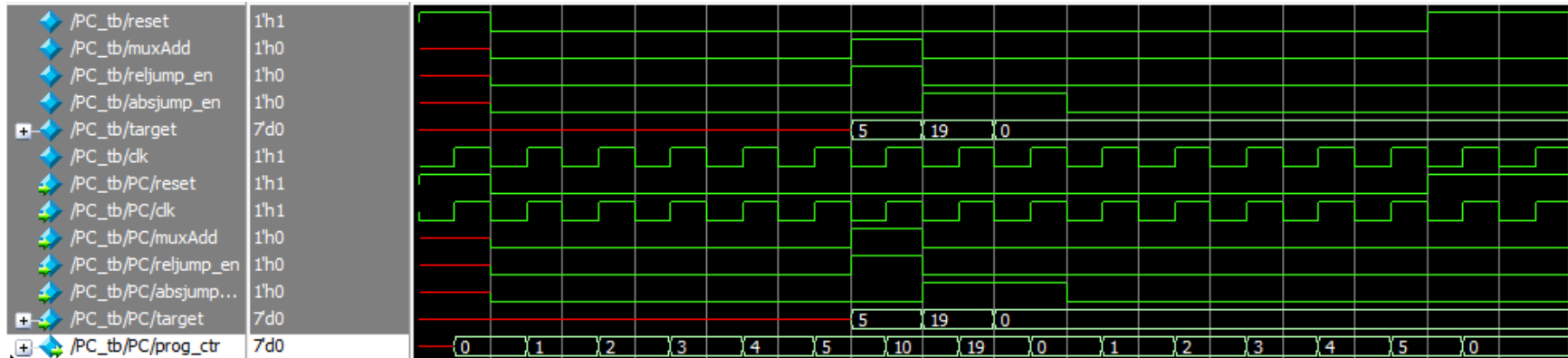
(Optional) Testbench Description

Our testbench tests the four main capabilities of the PC. We first test to ensure that the PC counter increments by one when no control signals are inputted. We then add the control signals to indicate that a bne instruction was called to check that the target value is added to the current PC. Next are two branch immediate tests, one being to a farther address, and one being a pseudo reset to 0. Finally we check that the reset flag resets the PC to 0.

Schematic



(Optional) Timing Diagram



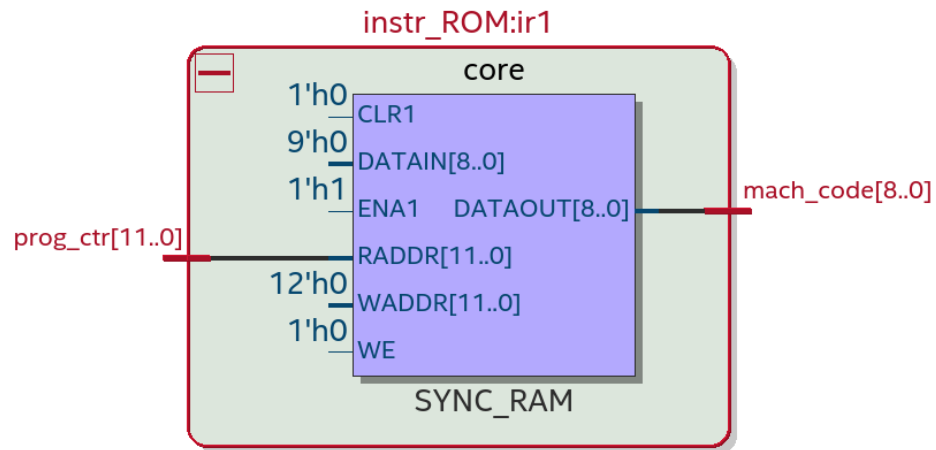
Instruction Memory

Module file name: instr_ROM.sv

Functionality Description

This module takes the machine code instructions that match the respective PC value in order to pass instructions into the circuit. After inputting the PC, the module returns the machine code found in the PC's position within the machine code text file where our machine code for our assembly programs is written.

Schematic



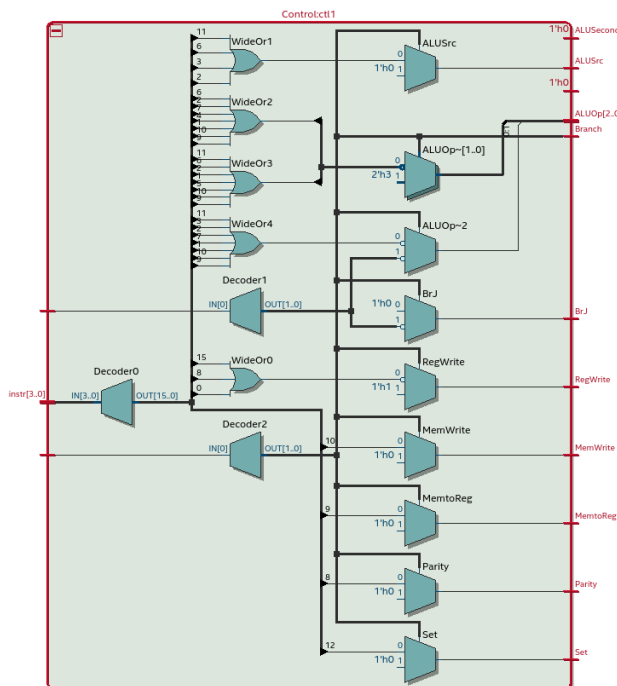
Control Decoder

Module file name: Control.sv

Functionality Description

The Control module receives the 5 most significant bits from our machine code and generates control signals needed by muxes to determine pathways as well as the ALU operation. For j and bne, we only need the first 2 bits to distinguish jump instructions. For our type N and R instructions, each unique 4 bit opcode is translated into turning output control signals either off or on. The possible control signals we have are RegDst, Branch, MemtoReg, MemWrite, ALUSrc, RegWrite, Set, Parity, BrJ, and ALUSecnd. The function of each control signal can be seen in the Architectural Overview.

Schematic



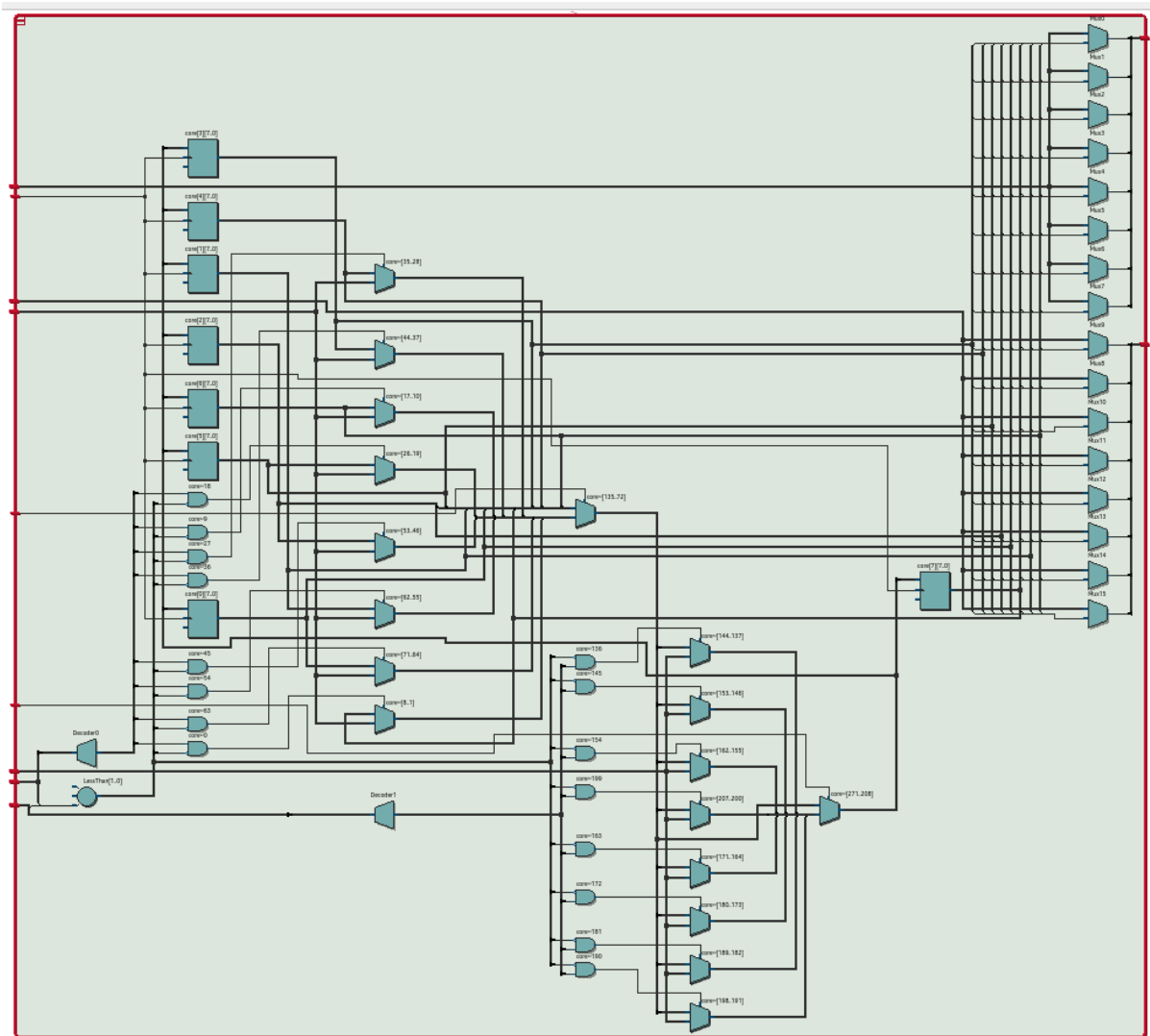
Register File

Module file name: reg_file.sv

Functionality Description

The Register File contains all of the data of the registers. It takes two inputs as well as control signals and outputs two data outputs. These inputs and outputs could be either register numbers or the data contained within a register depending on the control signals. The module can also both read and write to a designated register. Our register file has 16 registers all 8 bits wide. For our processor, we have dedicated the last register to store the parity word after the par instruction is called, so this register should not be overwritten.

Schematic



ALU (Arithmetic Logic Unit)

Module file name: alu.sv

Module testbench file name: ALUtestbench.sv

Functionality Description

The ALU performs almost all of our bitwise operations and arithmetic operations needed in our ISA. It receives two input data values and an ALU instruction in the form of control signals in order to perform the desired calculations. We also have the capability to output two values for the instruction lrsr which combines two registers and logical right shifts across both registers. If the 2nd result is not needed, it defaults to 0.

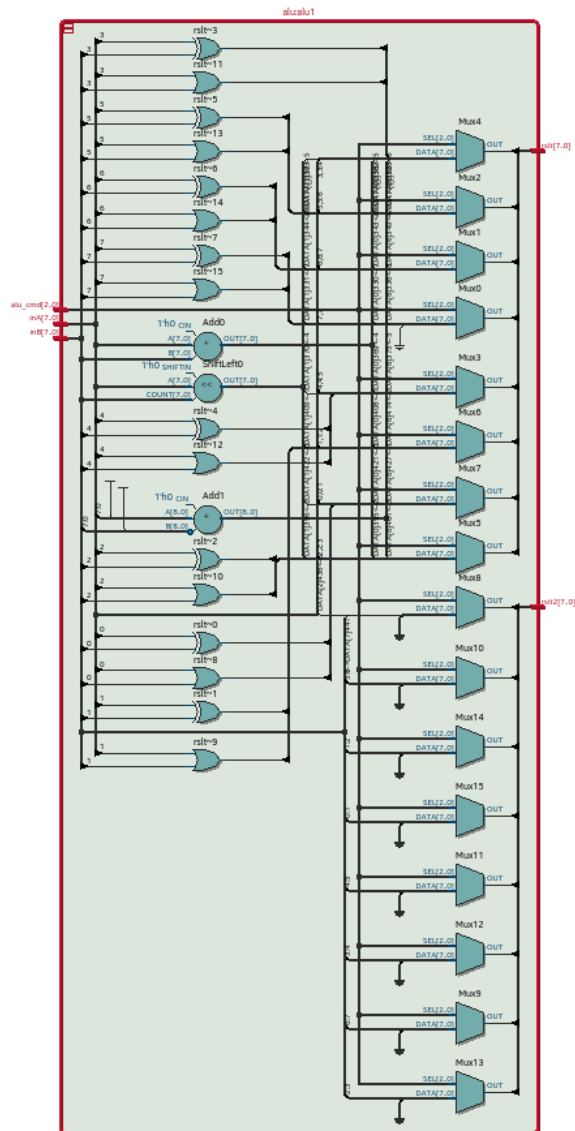
(Optional) Testbench Description

Our testbench tests the ALU by inputting ALU instructions and 2 input data values into the ALU. We can check the result outputs in the waveform viewer to ensure that our desired instruction has executed correctly.

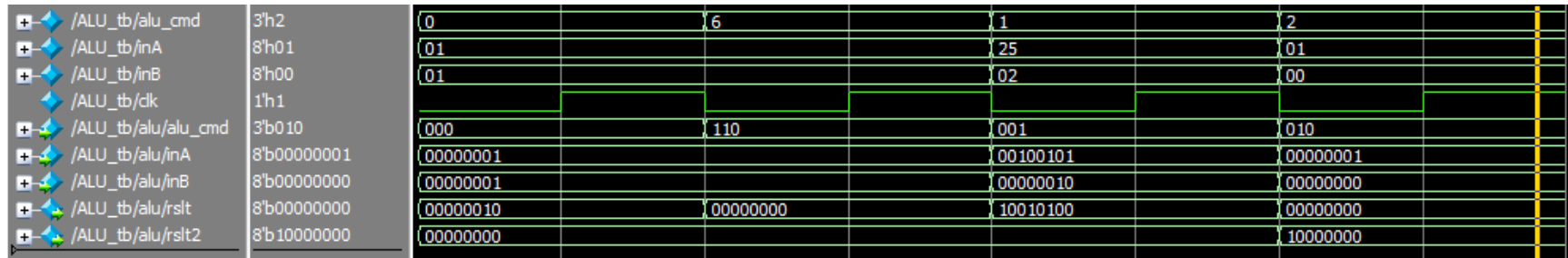
ALU Operations

We are demonstrating the add, subtract, left shift, and right shift commands. In our ISA, add is used for add and addi, subtract for subi, left shift for lsl, and right shift for lrsr.

Schematic



(Optional) Timing Diagram



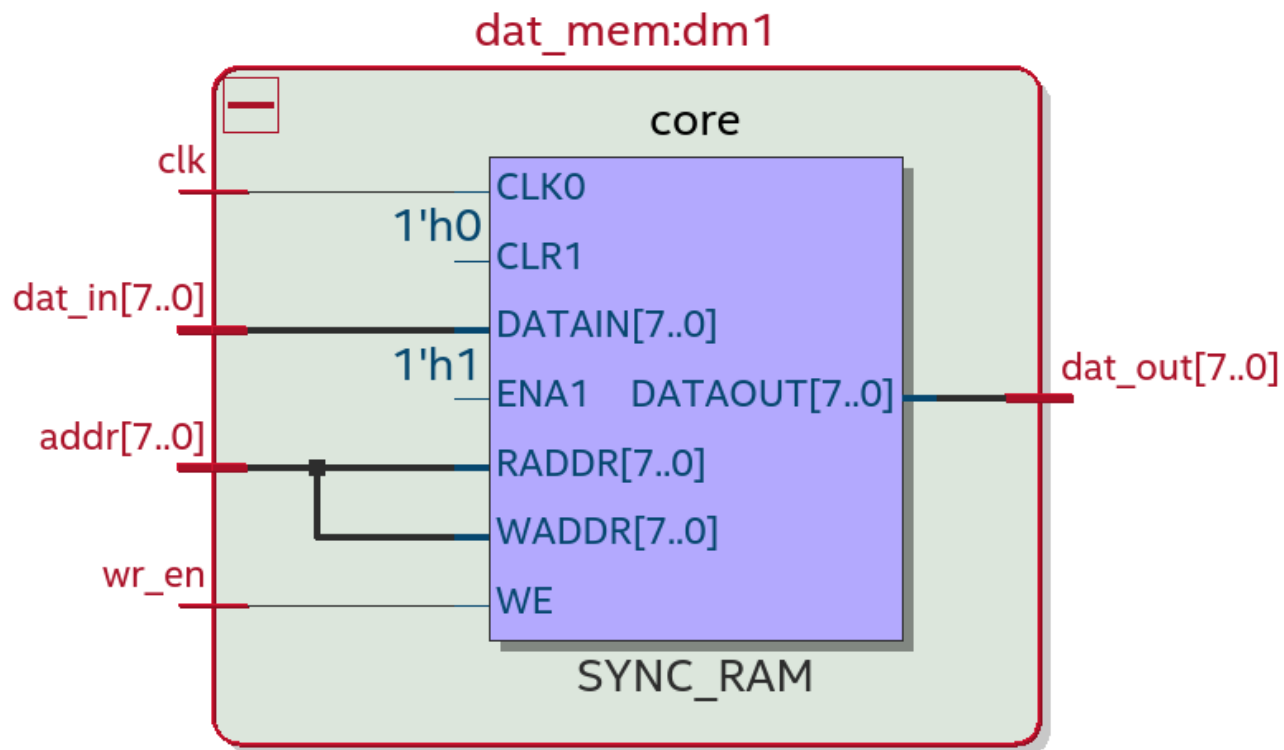
Data Memory

Module file name: dat_mem.sv

Functionality Description

The Data Memory module can write and read data memory entries. Our file allows for 256 different entries each 8 bits wide. The outputs of our three programs will be stored here.

Schematic



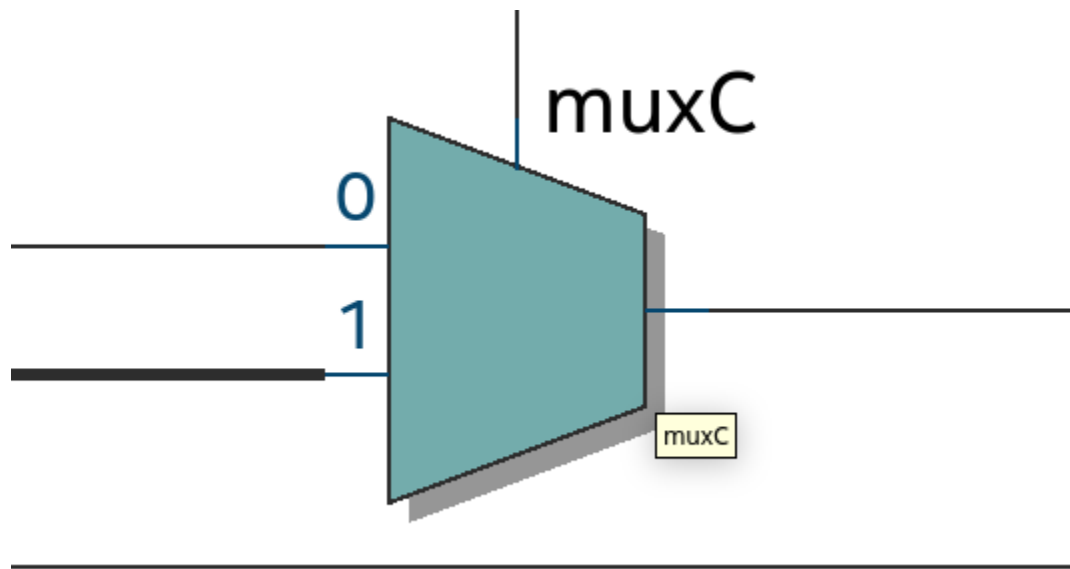
Muxes (Multiplexers)

Module file name: MuxC

Functionality Description

This Mux is used to select between data memory data or ALU output to be stored to the register file.

Schematic

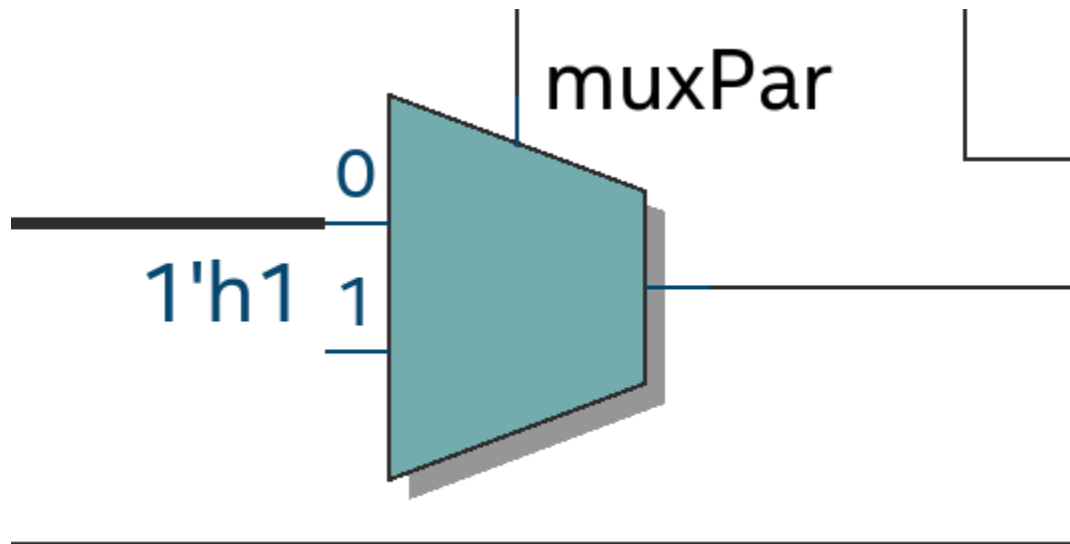


Module file name: muxPar

Functionality Description

This mux is used to select between register 15 when parity instruction is called, or pass through the value of set.

Schematic

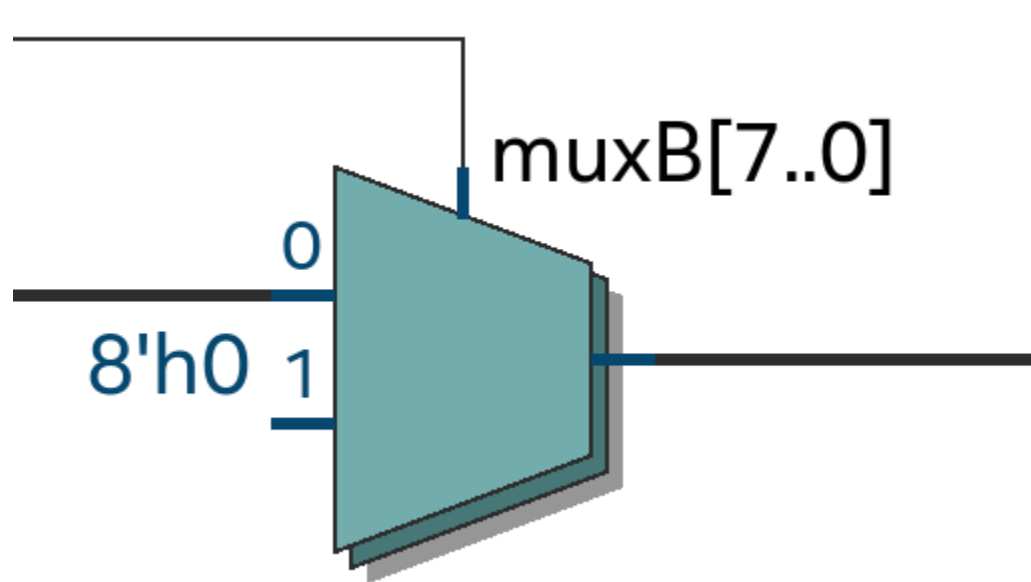


Module file name: muxB

Functionality Description

This mux is used to select between using data from a register file or the immediate in the machine code.

Schematic



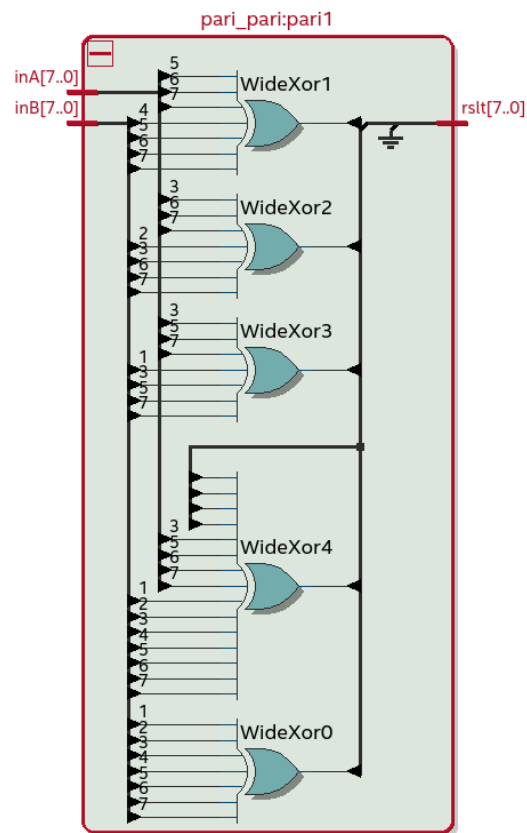
Parity

Module file name: pari_pari.sv

Functionality Description

This module calculates the parity word and stores it to register 15.

Schematic



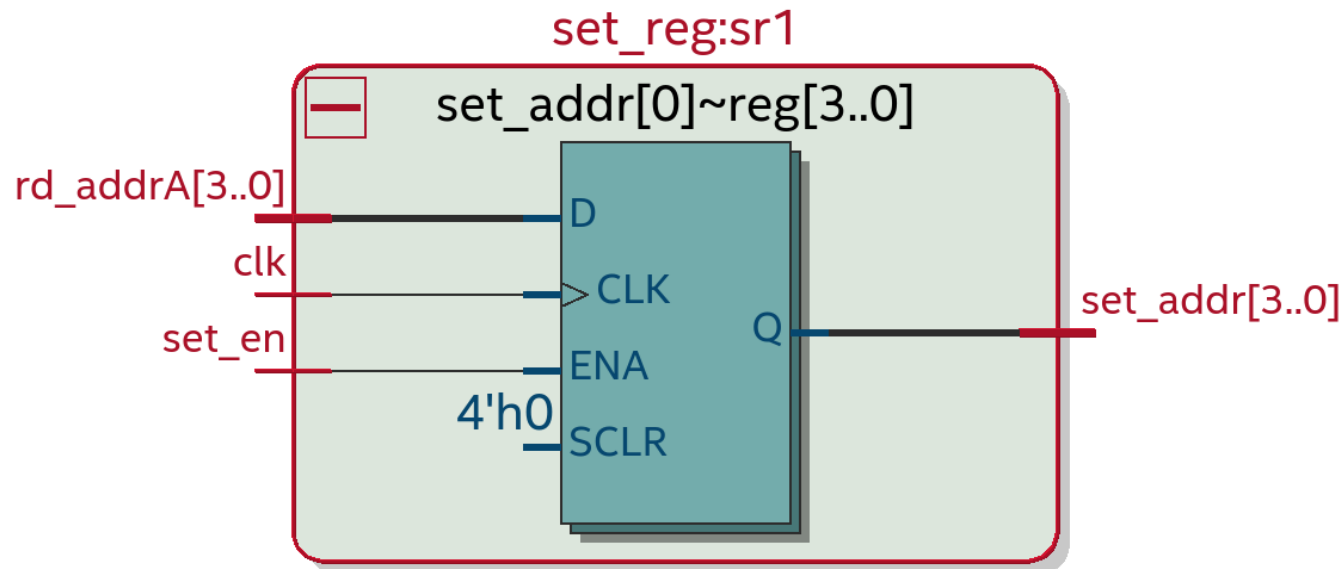
Set_Reg

Module file name: set_reg.sv

Functionality Description

This module sets the RA address to the input register number. This module has to be used before any other instruction as an RA value must be set.

Schematic



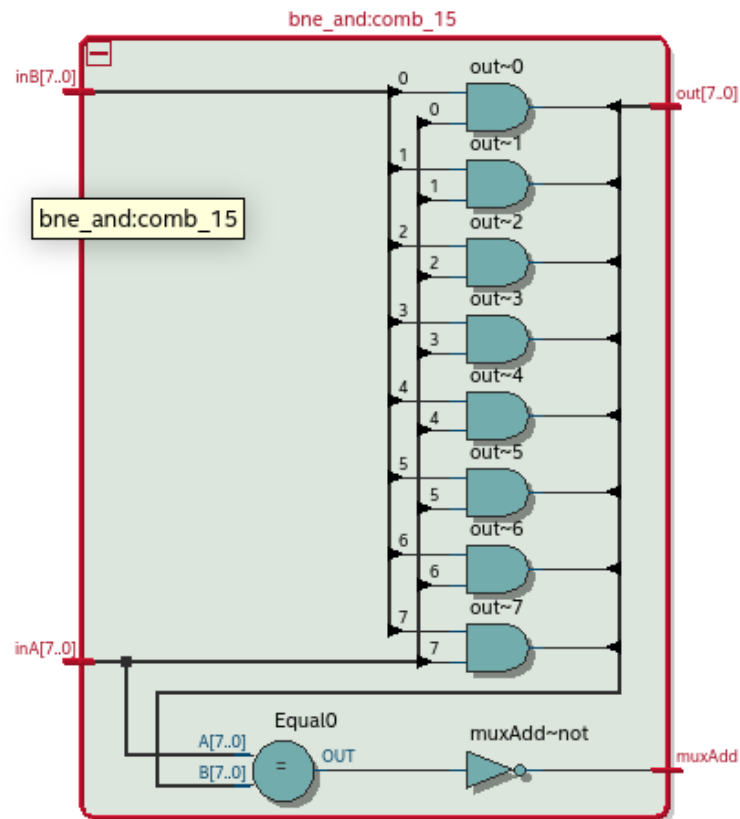
BNE_And

Module file name: bne_and.sv

Functionality Description

This module determines if two inputs are equal. This is used to generate a control signal indicating that the two inputs are not equal.

Schematic



Assembler

Functionality Description

This module converts the assembly code that a programmer writes to machine code that our processor can use. It also removes comments and fills in branch and jump values into the machine code.

Example

```
add R0 = 000010000
set R1 = 011000001
bne R1 7 = 110001111
```

6. Program Implementation

Program 1 Pseudocode

```
For (int i = 0; i < 30; i += 2) {
    Data_mem[30+i] = data_mem[i] << 4; //shift (b4 - b1) to front of data_mem[30+i](low bits)
    Data_mem[30+i+1] = data_mem[i+1] << 5 + data_mem[i][7:4] << 1; //shift b11-b5 to front of data_mem[30+i+1](high bits)
    P8 = ^(data_mem[30+i+1][7:1]); //calculate parity bit 8
    Data_mem[30+i+1] += P8; //add p8 to data_mem[30+i+1](high bits)
    P4 = ^(data_mem[30+i+1][7:4] , data_mem[30+i][7:5]); //calculate parity bit 4
    P2 = ^(data_mem[30+i+1][7:6, 3:2] , data_mem[i][3:2, 0]); //calculate parity bit 2
    P1 = ^(data_mem[30+i+1][7,5,3,2] , data_mem[i][3, 1:0]); //calculate parity bit 1
    P0 = ^(data_mem[i+1][2:0] , data_mem[i][7:0], P8, P2, P1,P0); //calculate parity bit 0
    Data_mem[30+i][4] =p4; //set p4 into low bit 4
    Data_mem[30+i][3] =data_mem[i][0]; //set
    Data_mem[30+i][2] =p2; //set p2 into low bit 2
    Data_mem[30+i][1] =p1; //set p1 into low bit 1
    Data_mem[30+i][0] =p0; //set p0 into low bit 0
}
```

```
}
```

Program 1 Assembly Code

```
    set  R2
    movi 0
    set  R4
    movi 15
    addi 15
    addi 2
    set  R5
    movi 0
    set  R4
    bne  R5 done_j1
    jump notdone_j1
done_j1:
    jump done
notdone_j1:
    set  R0
    lw   R2
    set  R3
    addi 1
    set  R1
    lw   R3
    set  R1
    par  R0           // parity instruction to output parity string to R5
    jump first
loop:
    set  R5
    movi 0
    set  R4
    bne  R5 done_j2
    jump notdone_j2
done_j2:
    jump done
notdone_j2:
    set  R2
    addi 1
    set  R0
    lw   R2
    set  R3
```

```

    addi 1
    set  R1
    lw   R3
    set  R1
    par  R0

first:
    set  R8
    add  R1
    lsl  5
    set  R6
    movi 8          // 1000
    lsl  1
    or   R15        // get p8
    lsr  R10
    lsr  R10
    lsr  R10
    lsr  R10
    set  R11
    add  R0
    lsl  1
    lsr  R10
    lsr  R10
    lsr  R10
    lsr  R10
    lsr  R10
    lsl  1
    set  R3
    add  R11
    add  R6
    set  R2
    addi 15
    addi 15
    addi 1
    sw   R3          // upper bit taken care of
    subi 15
    subi 15
    subi 1
    set  R11
    lsr  R10
    set  R3
    movi 0
    add  R11
    set  R6
    movi 8          // 00001000

```

```

or R15                // get p4
set R3
add R5
set R1
movi 0
add R0
lsl 7
lshr R10
lshr R10
lshr R10
lshr R10
set R6
movi 4                // 00000100
or R15                // get p2
set R3
add R6
add R1
set R6
movi 2                // 00000010 get p1
or R15
set R3
add R6
movi 1                // get p0
or R15
set R3
add R6
set R2
addi 15
addi 15
sw R3                // upper bit taken care of
subi 15
subi 15
addi 2
jump loop
done:
halt

```

Program 1 Machine Code

011000010
010110000
011000100
010111111
000101111
000100010
011000101
010110000
011000100
110101001
100001100
101101101
011000000
010010010
011000011
000100001
011000001
010010011
011000001
010000000
100100101
011000101
010110000
011000100
110101001
100011011
101101101
011000010
000100001
011000000

010010010
011000011
000100001
011000001
010010011
011000001
010000000
011001000
000010001
001100101
011000110
010111000
001100001
001011111
001111010
001111010
001111010
001111010
011001011
000010000
001100001
001111010
001111010
001111010
001111010
001111010
001100001
011000011
000011011
000010110
011000010
000101111

000101111
000100001
010100011
000111111
000111111
000110001
011001011
001111010
011000011
010110000
000011011
011000110
010111000
001011111
011000011
000010101
011000001
010110000
000010000
001100111
001111010
001111010
001111010
001111010
011000110
010110100
001011111
011000011
000010110
000010001
011000110
010110010

```

001011111
011000011
000010110
010110001
001011111
011000011
000010110
011000010
000101111
000101111
010100011
000111111
000111111
000100010
100010101
000000000

```

Program 2 Pseudocode

```

For (int i = 30; i < 59; i += 2){
    P8 = ^(data_mem[i+1][7:1])           //XORing every bit from 7 to 1 (b11:b5)
    P4 = ^(data_mem[i+1][7:4], data_mem[i][7:5]) //XORing (b11:b8) and (b4:b2)
    P2 = ^(data_mem[i+1][7,6,3,2], data_mem[i][7,6,3]) //XORing b11,10,7,6,4,3,1
    P1 = ^(data_mem[i+1][7,5,3,1], data_mem[i][7,5,3]) //XORing b11, 9, 7, 5, 4, 2, 1
    P0 = ^(data_mem[i+1][7:0], data_mem[i][7:1]) //Xoring every value
    $R7 = 0 //7th register for figuring out which is bad
    &R7[3] = P8^data_mem[i+1][1] //XORing regenerated with parity 8
    &R7[2] = P4^data_mem[i][4] //XOrRing regenerated with parity 4
    &R7[1] = P2^data_mem[i][2] //XORing regen with parity 2
    &R7[0] = P1^data_mem[i][1] //XORing regen with parity 1

    If (R7 > 8){ // Conditional for if R7 is greater than 8

```

```

        R7 = R7 - 7                                //Setting R7 to index for upper byte
        If (R7 == 1) break                          //Break if R7 is where parity bit is
        data_mem[i+1-30][R7] = 1 - data_mem[i+1][R7] //Else, flip
        R7 = R7 + 7                                //Restore for future logic
    }
    Else{                                           // If R7 is less than 8 (lower byte)
        If (R7 == 0 or 1 or 2 or 4) break          //Break if R7 is where parity bit is
        data_mem[i-30][R7] = 1 - data_mem[i][R7]   //Else, flip
    }
    Data_mem[i+1] >>                               //Right shift upper byte to get rid of p8
    Data_mem[i][4] = Data_mem[i][3]                // Get rid of p4, replace with b1
    for(int j =0; j < 5; j++){                      // Loop for shiting 4 times to puts bit back in place
        Data_mem[i]>>                               // Right shift for lower byte, opens msb
        Data_mem[i][7] = Data_mem[i+1][1]          // Carry lsb upper byte to msb lower byte
        Data_mem[i+1]>>                             // Right shift for upper byte
    }
    Data_mem[i-30] = Data_mem[i]
    Data_mem[i-29] = Data_mem[i+1]
}

```

Program 2 Assembly Code

```

Set R2
Movi 15                                // highest value that can be moved by immediate
Addi 15                                // making R2 to be 30
Set R4
Movi 15
Addi 15
Addi 15
Addi 15                                // making R4 to be 60
Set R3                                // Getting R3 (counter), to be 30 (R2)
Movi 0

```

```

Add R2
Loop:
  Set R0
  Lw R3
  Addi 1
  Set R1
  Lw R3
  Set R3
  Subi 1
  Set R7
  Movi 0
  Add R1                                // dealing with sets to have r7 = r1, and then place its parity in r7
  Par R0                                // get parity string
  Movi 0
  Add R15                               // having R7 be R15 (parity string)
  Lsr R7                                // shifting left to get parity index
  Set R6
  Movi 0
  Set R12
  Movi 0
  Add R7                                // having R12 save R7
  Subi 9                                // All values <= 8 would be negative
  Lsr R12                               // 7 shifts to see if its negative ....
  Lsr R12
  Lsr R12
  Lsr R12
  Lsr R12
  Lsr R12
  Lsr R12

  bne R6 bne1                          // conditional if statement to keep it within the msw
  jump not_bne1

```

```

bne1:
    jump else
not_bne1:
    Set R7
    Subi 7                // subtract to get value of >8
    Set R6
    Addi 1
    bne R7 bne2           // If r7 is not equal to parity it, go beyond jump
    jump not_bne2
    bne2:
    jump InFor
not_bne2:

    Jump for1            // Else we don't need to flip anything
InFor:
    Set R6
    lsl 1                // Loop for getting the right bit to flip (increments until R7 is 0
    Set R7
    subi 1               // Decrement of R7 for each time shifted left
    Set R13
    Movi 0

    bne R7 bne3          // If R7 is not 0 yet, we can still do the loop, else we xor
    jump not_bne3
bne3:
    jump InFor
not_bne3:
    Set R1
    Xor R7               // Xoring R1 and R7, to get the specific bit to flip
    Jump for1           // Going beyond the else branch
Else:
    Set R6

```

```
    Movi 4
    bne  R7 bne4
    jump not_bne4
```

// If R7 is not 0 yet, we can still do the loop, else we xor

```
bne4:
    jump For1
not_bne4:
    Movi 2
    bne  R7 bne5
    jump not_bne5
bne5:
    jump For1
not_bne5:
```

```
    Movi 0
    bne  R7 bne6
    jump not_bne6
bne6:
    jump For1
not_bne6:
```

```
    Movi 1
    bne  R7 bne7
    jump not_bne7
bne7:
    jump For1
not_bne7:
inFor2:
```

```
    set R6
    lsl 1
    Set R7
    Subi 1
    bne  R13 bne8
```

// shifts R6 left until R7 is 0

// If not 0 yet (R13), keep looping, else xor

```

        jump not_bne8
bne8:
        jump InFor2
not_bne8:
        Set R0
        Xor R7                // Xoring R0 and R7, to get specific bit
For1:
        Set R1
        Lsr R13                // Shift right into garbage register, get rid of parity bit
                                // used to isolate b1 (get rid of parity bits)
        Set R6
        Movi 0
        Add R0                // Having R6 = r0 (lsw)
        Lsl 4
        Lsr R13                // Shift right into garbage register (r13) 7 times :(
        Lsr R13
        Lsr R13
        Lsr R13
        Lsr R13
        Lsr R13
        Lsr R13                // b1 is now isolated
        Set R1
        Lsr R0                // Shift right r1 to r0 4 times (which decodes)
        Lsr R0
        Lsr R0
        Lsr R0
        Set R0
        Lsr R13                // Shift right r0 to garbage register (r13) to get rid of parity bit
        Lsl 1                  // Shift left to open lsb to R6
        Add R6                // Get R6 to lsb of R0
        set R3
        subi 15
        subi 14

```



```

    set R0
    Sw R3                // Storing to memory
    set R3
    subi 1
    Set R1
    sw R3
    Set R3
    Addi 15
    Addi 15
    Addi 1
    bne  R4 bne9          // If i is not 59 yet (end)
    jump not_bne9
bne9:
    jump loop
not_bne9:
    Done:
    halt

```

Program 2 Machine Code

```

011000010
010111111
000101111
011000100
010111111
000101111
000101111
000101111
011000011

```

[illegible]

011000111
000110111
011000110
000100001
110111001
100110000
100110001
101010111
011000110
001100001
011000111
000110001
011001101
010110000
110111001
100111010
100110001
011000001
001000111
101010111
011000110
010110100
110111001
101000010
101010111
010110010
110111001
101000110
101010111
010110000
110111001
101001010

101010111
010110001
110111001
101001110
101010111
011000110
001100001
011000111
000110001
111101001
101010101
101001110
011000000
001000111
011000001
001111101
011000110
010110000
000010000
001100100
001111101
001111101
001111101
001111101
001111101
001111101
001111101
011000001
001110000
001110000
001110000
001110000

011000000
001111101
001100001
000010110
011000011
000111111
000111110
011000000
010100011
011000011
000110001
011000001
010100011
011000011
000101111
000101111
000100001
110100001
101111101
100001011
000000000

Program 3 Pseudocode

partAOutput = 0	//data mem [33]
partBOutput = 0	//data mem [34]
partCOutput = 0	//data mem [35]
Bool sameByte = 0	//Used for part A and B to account for byte boundaries
String message = data_mem[0:31];	//Input string
For int Byte_num = 0; Byte_num < 31; Byte_num++ {	//checking all but last byte

```

$R1 = data_mem[Byte_num];
$R2 = data_mem[Byte_num + 1];
sameByte = 0
For int i = 0; i < 8; i++){
    if($R1[0:5] XOR data_mem[32] [7:3]){
        partCOutput++;
        If (sameByte == 0){
            partBOutput++;
            sameByte = 1;
        }
        If (i < 5){
            partAOutput++;
        }
    }
    $R1 << 1;
    $R1[7] = $R2[0];
    $R2 << 1;
}
$R1 = data_mem[31];
sameByte = 0;
For int i = 0; i < 4; i++){
    if($R1[0:5] XOR data_mem[32] [7:3]){
        partCOutput++;
        partAOutput++;
        If (sameByte == 0){
            partBOutput++;
            sameByte = 1;
        }
    }
    $R1 << 1;
}
}

```

```

//Load each byte starting from byte 0
//Load the byte that follows the byte in R1 into R2
//Indicates start of new byte for part a and b

//partCOutput doesn't care about byte boundaries
//partBOutput should only at most increase once per byte

//partAOutput cares about byte boundaries
//pattern can only occur in the first 4 bits in the byte

//shift R1 left by one bit
//carry over first bit in R2 into R1
//shift R2 left by one bit

//loop to check final byte

//no more byte boundary for partAOutput and partCOutput

//partBOutput should only at most increase once per byte

//shift R1 left by one bit

```

Program 3 Assembly Code

Program 3 ASM pseudocode (CONVERT INTO OUR ISA BEFORE TURNING IN, **DOUBLE CHECK FOR ERRORS**)

Registers:

R0: address for current byte

R1: high byte

R2: low byte

R3: mem[32] (shift right 3 so lower 5 bits are string to compare), match result

R4: count for a

R5: count for b

R6: count for c

R7: found in current byte?

R8: bits processed

R9: zero

R10: scratch compare value

R11: 4 (condition a sentinel)

R12: 32 (end-of-string sentinel)

R13: 8 (end-of-byte sentinel)

Memory:

mem[0-31]: message bytes

mem[32][7:3]: string to search

mem[33]: count for a

mem[34]: count for b

mem[35]: count for c

movi 0	// R0 = 0, start from byte 0
set R2	// set RA to R2 (low byte)
lw R0	// load first byte into R2

set R0	// set RA to R0 (current byte address)
addi 1	// increment current memory address
set R1	// set RA to R1 (high byte)
lw R0	// load second byte into R1

set R4	// set RA to R4
movi 0	// R4 = 0, a count 0
set R5	// set RA to R5
movi 0	// R5 = 0, b count 0
set R6	// set RA to R6
movi 0	// R6 = 0, c count 0
set R7	// set RA to R7
movi 0	// R7 = 0, not found in current byte
set R8	// set RA to R8
movi 0	// R8 = 0, 0 bits processed
set R9	// set RA to R9
movi 0	// R9 = 0, zero register

set R11	/ set RA to R11 (condition a sentinel)
movi 4	

set R12	// set RA to R12 (end-of-string sentinel)
movi 7	// R12 = 7 (goal 32)
addi 7	// R12 = 14 (goal 32)
addi 7	// R12 = 21 (goal 32)
addi 7	// R12 = 28 (goal 32)
addi 4	// R12 = 32 (goal 32)

set R13	// set RA to R13 (end-of-byte sentinel)
movi 7	// R13 = 7 (goal 8)
addi 1	// R13 = 8


```

loop:  set R3
      lw R3          // R3 = mem[R3] = mem[32]

      lsr R9          // R3[7:3] -> R3[4:0]
      lsr R9          // ...
      lsr R9          // ...

      xor R2          // R3 = R3 XOR R2, checking for R3[5:0]
      lsl 3           // clear topmost 3 bits of R3
      lsr R9          // R3[7:3] -> R3[4:0]
      lsr R9          // ...
      lsr R9          // ...

      bne R9 isz      // if R3 != 0, no match, branch to isz
      movi 1          // else set R3 to 1
      bne R9 notz     // and then branch to notz
isz:   movi 0          // if match, set R3 to 1
notz:

      set R8          // set RA to R8 (bits processed)
      addi 1          // increment bits processed by 1
      set R11         // set RA to R11 (condition a sentinel)
      bne R8 skip1    // if processed 4 bits, do not check a
      set R4          // set RA to R4 (count for a)
      add R3          // R4 = R4 + result

skip1: set R10         // set RA to R10 (compare value)
      movi 1          // R10 = 1
      set R7          // load if found in current byte
      bne R10 skip2   // if found in current byte, do not check b
      add R3          // R7 = R7 + result
      set R5          // set RA to R5 (count for b)
      add R3          // R5 = R5 + result

```

```

skip2:  set R6                // set RA to R6 (count for c)
        add R3                // R6 = R6 + result
        set R0                // set RA to R0 (current byte address)
        addi 1                // increment byte address
        set R12               // set RA to R12 (end-of-string sentinel)
        bne R0 skip3          // done check 1: all bytes processed?
        set R13               // set RA to R13 (end-of-byte sentinel)
        bne R8 skip4          // done check 2: all bits in last byte processed?
        j done                // done

skip3:   set R13                // set RA to R13 (end-of-byte sentinel)
        bne R8 skip4          // all bits in current byte processed?
        set R1                // set RA to R1 (high byte)
        lw R0                 // load new byte if all bits processed

skip4:   j loop

done:    set R12                // set RA to R12 (reusing as memory address)
        addi 1                // R12 = 33 (count a result address)
        set R4                // set RA to R4 (a count)
        sw R12                // store a count in mem[33]
        set R12               // set RA to R12 (reusing as memory address)
        addi 1                // R12 = 34 (count b result address)
        set R5                // set RA to R5 (b count)
        sw R12                // store b count in mem[34]
        set R12               // set RA to R12 (reusing as memory address)
        addi 1                // R12 = 35 (count c result address)
        set R6                // set RA to R6 (c count)
        sw R12                // store c count in mem[35]
        halt                  // end of program, halt instruction

```

Program 3 Machine Code

010110000
011000010
010010000
011000000
000100001
011000001
010010000
011000100
010110000
011000101
010110000
011000110
010110000
011000111
010110000
011001000
010110000
011001001
010110000
011001011
010110100
011001100
010110111
000100111
000100111
000100111
000100100
011001101
010110111
000100001

011000011
010010011
001111001
001111001
001111001
001000010
001100011
001111001
001111001
001111001
111001010
010110001
111001001
010110000
011001000
000100001
011001011
111000010
011000100
000010011
011001010
010110001
011000111
111010011
000010011
011000101
000010011
011000110
000010011
011000000
000100001
011001100

110000011
011001101
111000101
101000111
011001101
111000010
011000001
010010000
100011110
011001100
000100001
011000100
010101100
011001100
000100001
011000101
010101100
011001100
000100001
011000110
010101100
000000000

7.Changelog

- Milestone 3
 - Individual Component Specification
 - Added Assembler description
 - Program Implementation
 - Added Machine Code versions of each program
 - Updated bugs in assembly code
- Milestone 2
 - Architectural Overview
 - added control logic, fixed incorrect branching, added labels
 - Individual Component Specification
 - Initial version
 - Programmer's Model [Lite]
 - Updated 4.1, added 4.3
 - Program Implementation
 - Updated all programs to match our ISA
- Milestone 1
 - Initial version